

ScalaLR

ScalaLR is a straightforward LR-parser generator for Scala that translates its own host notation (a description of a target notation) to the essential components of a parser for the target notation expressed in Scala. Its accompanying library provides implementations both of **Pull** and of **Push** parsing automata that are used (with the parsing tables it generates) to implement the target language parser. The former is designed for conventional “parse-to-completion” situations, the latter supports the engineering of incremental parsers whose state and intermediate results can be inspected “in flight”.

Host notation

The host notation for grammar productions and priorities is reminiscent of Bison’s notation; but there are important overall differences from Bison, as exemplified by the following fragment. We will document these in detail in due course.

```
%notation Expr
%package   expr.Expr

%include {
  // Scala source to be included in a generated file that supports or implements a lexer
  import org.sufrin.utility.SourceTextCursor
  import org.sufrin.scalalr.SourceLocation

  def Scanner(chars: SourceTextCursor): Scanner = new Scanner(chars)

  class Scanner(chars: SourceTextCursor) extends Iterator[Token] { ... }
}

%token ID(String) `(` `)` `[` `]` `` `;` LEXICALERROR(String) // $1
%left `+` // $2
%left `*` // $2

%rules

%include {
  // Scala source to be included in a generated file that supports or implements
  // the abstract syntax derived from productions

  import org.sufrin.scalalr.SourceLocation
  // Abstract syntax nodes $4
  trait Expr { val loc: SourceLocation } // $4
```

```

case class Id(s: String, loc: SourceLocation) extends Expr
case class Binop(op: String, l: Expr, r: Expr, loc: SourceLocation) extends Expr
case class Bra(expr: Expr, loc: SourceLocation) extends Expr
}

exprs: (List[Expr]) = expr          { List($expr) }           // $3, $4
                        |   exprs `;` expr { $expr::$exprs }   // $4
                        ;

expr: Expr = ID              { Id($ID, $START) }             // $4,
                        | l:expr `*` r:expr { Binop("*", $l, $r, $START) }
                        | l:expr `+` r:expr { Binop("+", $l, $r, $START) }
                        | "(" expr ")"      { Bra($expr, $START) }           // $5
                        | `[` expr `]`      { $expr }
                        ;

```

1. Tokens (terminal symbols) may be specified. Each that carries an irredundant value must have the type of that value specified.
2. Shift-reduce conflicts can be resolved by specifying the associativity and precedence of (terminal) symbols, as in Bison.
3. Nonterminal symbols have types specified explicitly on the left hand side of their definition.
4. The abstract syntax node represented by each production is specified as a Scala block expression at its end. Such expressions may refer to the values of symbols (terminal or nonterminal) that appear in the production, by `$label` (for a symbol labelled in the production by prefixing it with `label:`), or by `$symbol` when that `symbol` appears unlabelled and uniquely. They may also refer to the start and end source location of the text matched by the production using `$START` and `$END`.
5. Tokens enclosed in single quotes, double quotes or backticks are treated identically during code generation: they need not be declared in a `%token` section.

The generated target language code can easily be incorporated into a production Scala program. Here's a test of the earlier example that uses the `Pull` automaton.

```

object runexpr {

  import expr.Expr.Components
  import expr.Expr.Scanner._

  import org.sufrin.utility.PrettyPrint._
  import org.sufrin.utility._

  def main(args: Array[String]): Unit = {

```

```

val source = """a; a+b; a*b+c*d*(e+f)*[g+h]; p+q*r"""
val scanner = Scanner(SourceTextCursor(source))

def next(): Token = if (scanner.hasNext) scanner.next() else $end

val parser = LRParser.Pull[Token](Components)(scanner.sourceLocation)
parser.run(next).prettyPrint()
}
}

```

Documentation is evolving but for the moment it should be sufficient for a knowledgeable reader to inspect the source texts of the notation specifications and driver programs to be found within directories nested with **examples**.

Implementations

Grammar analysis **ScalaLR** uses **GNU Bison (version 3.8.2)** as an internal workhorse to compute the LR shift-reduce parser tables for the grammar (rules) of the source notation, and to do any necessary detailed diagnostics on the grammar.

Production-quality Implementations There are (now: late April 2026) several stable production quality implementations of the program, bootstrapped from an original handwritten parser and a simple code generator.

1. The **scalalrlifeboat** program uses the original bootstrap handwritten parser and the original bootstrap code generator. Perhaps we should have called it **scalalrboot**...

Usage: **scalalrlifeboat** [--output=<outputpath>] [<file> ...]

2. The **scalalrgen** program is described in the **COMMANDLINE** documentation. It would be an exaggeration to say that its parser is completely self-hosting, but it offers a choice of parsing with the original bootstrap parser, and a parser for compatible syntax generated with **scalalr** technology (known as **FLaB** – first language atop bootstrap).

Usage: **scalalrgen** [-flab | -boot | -h | [--output=<outputpath>] [<file> ...]

Experimental Implementations Experience with using earlier versions of **scalalr** demonstrated a rather high incidence of noisy errors with a single trivial cause: the omission of just one semicolon between rule definitions that was required by the earlier host notation.

This has been corrected in later versions of the host notation and is implemented right now in the **slab** and **slabslab** processors.

The latter is the first in the entire sequence that is self-hosting: in the sense that its input language can be described *in* its input language, and parsed by a

parser whose parsing components it generated itself.

3. The most recent *experimental* implementations of the program can be found in the **slab** module and the **SLABEXPERIMENTS/** directory.

Generated files

When all is well, the generator produces several components (in distinct files) from each parser specification; these appear in the directory corresponding to `%path` specification, and are named:

1. **Scanner.scala** defines the Scala case classes and types corresponding to each `%token` definition. There is provision for including the source text of the lexical scanner in this file.
2. **Tables.scala** defines the shift-reduce tables corresponding to the grammar. These will be interpreted by an LRParser automaton at parse-time.
3. **Reduction.scala** defines a function that maps each production number to a function that combines the values that are generated by its right-hand-side to form the value of the production itself.
4. **Components.scala** defines a structure that incorporates the semantics of the Tables and Reduction files.

These files are generated in three phases:

1. The generator produces a plain Bison grammar `.y` file in which all grammar tokens enclosed in quotes have been transformed to straightforward Bison names of the form **TOK-*nn*** in order to avoid confusing Bison. Under normal circumstances one need not inspect the `.y` file, and the diagnostic/report files are expressed as they were in the Scalalr source.
2. Bison is run to generate the LR parse tables as well as report and diagnostic files, all named for the specified `%notation`. The `.output` and `.html` files contain identical information: a report on the notation with details of states, conflicts, etc. The `.log` file provides (when appropriate) counterexamples illustrating conflicts in the grammar. These are intended to be helpful in diagnosing conflicts/ambiguities that Bison discovers while processing the grammar specification.
3. The `.xml` file output by Bison contains, among other information, an encoding of the information Scalalr now uses to generate its **Tables** and **Reduction** files.

Further Reading

1. The best-documented simple example of a program that uses *scalalr*-generated parsing components appears in the **LIFEBOAT/** directory in `runtinyfun.scalalr`.

2. Some cursory tests that were used in building the initial bootstrap appear in `bootstrap/src/test`; they generate(d) components in the `testbed` module's `src/test/scala` directory.
3. The file `Bootstrapping.md` provides an explanation of the self-hosting bootstrap stages.

LR Parsing Conflicts

A few notation definitions that result in shift-reduce or reduce-reduce conflicts are gathered (as embedded strings) in the `App` defined in `bootstrap/src/test/scala/genconflicts.scala`. This can be run to test the reporting of such conflicts. Each example generates a log (as well as the expected generated files) in `testbed/src/test/conflicts`.

Gotchas

1. **Error recovery** is not yet properly implemented. The `Pull` automaton reports the first syntax error and bails by throwing an exception. The `Push` automaton does likewise if there is no `error`-handling state available (see Bison documentation for an explanation of the `error` virtual token), and the “recovery” that otherwise results is not properly implemented or documented. Despite this it is straightforward to construct an REP-type interface that appears to recover from syntax errors. For an example, see `runtinyfun`.
2. **Trivial typos** are problematic. Experience with using the languages early in the bootstrap sequence demonstrated the high incidence of errors caused by the omission of semicolons between rules. This has been corrected in the `slab` notation and is implemented right now in the `slab` processor; and in the `slabslab` processor, the first in the entire sequence that is self-hosting.
3. **Scala code quotations** such as appear in `%include` passages and as action expressions need a little care. The normal form of a code quotation is a passage that opens with `{`, has properly-nested occurrences of `{` and `}` within it and ends with a closing `}` that matches the opening. **But** if an unmatched brace appears (for example in a character or string quote or in a comment) it can upset balance. **NB:** *Bison itself has the same requirement for its code inserts and quotations.* One solution is to use the alternative braces `«` and `»` to quote code. Another solution, “forcibly” balancing the quotation, is exemplified by the following extract from a code quotation defining the scanner for the `ScalaR` notation itself.

```
case '{' => // } to balance the code quotation
nextChar(); afterNextChar(CODE(chars.takeNested('{', '}') .mkString(")))
case '«' => // » to balance the code quotation
nextChar(); afterNextChar(CODE(chars.takeNested('«', '»') .mkString(")))
```

Roadmap

We aim to accomplish the following tasks as soon as we can. They are listed here in no particular order.

1. Error recovery properly implemented.
2. System to be self-hosting: ie using a scalar-derived parser rather than the present hand-coded recursive descent parser. [DONE]
3. Higher-level constructs such as %list, %option to express complex grammar expressions that would normally have to be “hand-coded” such as:

```
exprlist: List[Expr] = exprlistr { $exprlistr.reverse }
exprlistr: List[Expr] =
  expr { List($expr) } | exprlist1 ',' expr { $expr :: $exprlistr }
exprlist: List[Expr] =
  expr { List($expr) } | expr ',' exprlist { $expr :: $exprlist }
optexpr: Option[Expr] =
  { None } | expr { Some($expr) }
```

These could be expressed more concisely in-situ, for example:

```
ID '(' exprlist: (%revlist expr ',') ')' { Apply($ID, $exprlist) }
ID '(' exprlist: (%list expr ',') ')' { Apply($ID, $exprlist) }
RETURN optexpr: (%option expr) ';' { Return($optexpr) }
```

4. Build a new code generation module. This should be straightforward, and we expect to improve drastically on the structure, and functionality of the existing generator as we now have a parser derived from a definitive grammar and abstract syntax.

The published code will reflect current partial progress towards them when appropriate.

Working Assumption

ScalaLR was designed on the assumption the components it generates will become part of a parser that will yield an abstract syntax tree. *Of course there's no harm in the parser yielding (say) a numeric value, or even being run for its side effects and yielding **Unit** – though this may not be common.*

Valediction

I regret **the inscrutability of the bootstrap code-generator**. I have no excuse for this beyond my having wanted to prioritise fast turnaround while I was first experimenting with my approach. Some might say that the fact that it can **still** be used in a near production environment is testimony to my cunning, but I couldn't possibly comment!

BS: April 29th, 2026